



QSymGen

A QSpice DLL Symbol Generator

Revision Date: 2026.04.25

Background

Creating QSpice symbols is reasonably easy for DLL blocks with only a few pins. However, my micro-controller simulator project (QMdbSim) will need symbols with potentially dozens of pins for (hopefully) many different micro-controllers. Creating them manually is a roadblock.

And it's not just the raw pin count that poses a challenge – micro-controllers have tri-state pins (GPIO pins). QSpice has no native support for bi-directional DLL pins.

The only workable solution seems to be a GPIO symbol that exposes a tri-state pin to the schematic and has three pins on the C-Block side (input, output, and control). In other words, the DLL must have three pins for every micro-controller GPIO pin plus a GPIO symbol. Even for a simple 8-pin micro-controller, creating symbols is tedious. A 40-pin device with GPIO pins would quickly become overwhelming.

And even after the symbol is created, the DLL will need code to manage the GPIO pin states. It's largely boilerplate stuff but why not generate the code when generating the symbol?

Well, this is a first pass so it doesn't do all of that. As a “proof of concept,” it demonstrates using the QParser2 library to generate symbol files (*.qsym). Generating schematic files (*.qsch) follows the same general approach.

QSymGen

QSymGen is a command-line tool that reads a “pin definition file” and produces a QSpice symbol file. Currently, the only supported pin types are “input” and “output.”

Using QSymGen straight-forward: Create a text file with pin definitions, run `QSymGen.exe`, and it will produce a DLL symbol file. Drag the symbol onto a schematic and right-click to generate C-Block template code.

Run QSymGen with no parameters to get syntax information. See the example `PinDefs.txt` file for details about defining your own pin definitions files.

Limitations

The current version of the program is pretty basic:

- It supports only input/output pins and generates no DLL code. GPIO and code generation stuff is coming.
- The symbol file type is set as “`ø(.dll)`”. You can, of course, manually change it to a hierarchical or X-type symbol.
- The pin data type are hard-coded as 64-bit double (“Float”). You can, of course, modify the symbol manually or change/enhance/improve the source code.

Reminder: Once a symbol is dropped onto a schematic, changing the symbol file does not modify the symbol in the schematic. If you change the symbol, delete the symbol from the schematic and add it again.

Conclusion

I hope that you this tool useful. And, of course, let me know if you find problems or suggestions for improving the code or this documentation.

--robert